

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №5
по курсу «Разработка кроссплатформенных приложений»
на тему «Многопоточность в Java»

Выполнили студенты группы 23ВВИ2:

Кокарев Д.С.

Монин И.А.

Приняли:

Юрова О.В.

Карамышева Н.С.

Пенза 2026

Цель работы: научиться создавать многопоточные приложения с использованием стандартных средств языка Java.

Лабораторное задание: модифицировать приложение из предыдущей лабораторной работы, реализовав вычисление определенного интеграла в нескольких дополнительных потоках, снимая нагрузку с основного потока и предотвращая “подвисание” графического интерфейса. Реализовать многопоточность путем наследования от класса Thread.

Ход работы:

1. Создали класс IntegrationTask, унаследованный от Thread

```
1 package rcpa.labs.service;
2
3 public class IntegrationTask extends Thread{ 4 usages 1 i.monin
4     private final double startX; 2 usages
5     private final double endX; 3 usages
6     private final double step; 3 usages
7     private double partialSum = 0; 3 usages
8     private final boolean useTrapMethod; 2 usages
9
10    public IntegrationTask(double startX, double endX, double step, boolean useTrapMethod) { 1 usage 1 i.monin
11        this.startX = startX;
12        this.endX = endX;
13        this.step = step;
14        this.useTrapMethod = useTrapMethod;
15    }
```

Рисунок 1. Создание класса, унаследованного от Thread

2. Переопределили метод run(). Его реализация будет работать в рамках отдельного потока.

```
17 @Override 1 i.monin
18 public void run() {
19     double x = startX;
20     while (x < endX) {
21         double nextX = Math.min(x + step, endX);
22         if (useTrapMethod) {
23             partialSum += (nextX - x) * (Math.exp(-x) + Math.exp(-nextX)) / 2;
24         } else {
25             partialSum += Math.exp(-x) * step;
26         }
27         x = nextX;
28
29         if (Thread.interrupted()) {
30             return;
31         }
32     }
33 }
```

Рисунок 2. Переопределение метода run

3. Задаем количество потоков в соответствии с вариантом бригады.

```
83 //Потоки
84 int THREAD_COUNT = 5; 5 usages
85
```

Рисунок 3. Количество потоков

4. Изменили вычисления интегралов на многопоточный вариант, разбивая верхнюю и нижнюю границу на подэлементы.

```
274 private String integrationResult(double bottomBorder, double topBorder, double stepIntegration, 2 us
275                                 boolean useTrapMethod) {
276     double range = topBorder - bottomBorder;
277     double segmentSize = range / THREAD_COUNT;
278
279     IntegrationTask[] tasks = new IntegrationTask[THREAD_COUNT];
280
281     for (int i = 0; i < THREAD_COUNT; i++) {
282         double segStart = bottomBorder + i * segmentSize;
283         double segEnd = (i == THREAD_COUNT - 1) ? topBorder : bottomBorder + (i + 1) * segmentSize;
284
285         tasks[i] = new IntegrationTask(segStart, segEnd, stepIntegration, useTrapMethod);
286         tasks[i].start();
287     }
288
289     double totalSum = 0;
290     try {
291         for (int i = 0; i < THREAD_COUNT; i++) {
292             tasks[i].join();
293             totalSum += tasks[i].getPartialSum();
294         }
295     } catch (InterruptedException e) {
296         Thread.currentThread().interrupt();
297         return "";
298     }
299     return Double.toString(totalSum);
300 }
```

Рисунок 4. Добавление многопоточности

Вывод: в ходе выполнения лабораторной работы был получен опыт в создании многопоточных приложений с использованием стандартных средств языка Java.